《计算机操作系统》上机实验报告 EX1

实验 1 Linux 的基本操作

学号：待填写　　姓名：待填写　　班级：待填写　　实验日期：2026 年 11 月 10 日

一、实验目的

1．了解 Linux 的文件系统结构。

2．掌握 Linux 的基本操作命令。

3．掌握 nano、tar 和 gzip 等基本工具的使用。

4．熟悉 Linux 下的 C 语言编译链接环境。

二、实验环境

PPT 给定环境为 Ubuntu 12.4（32 位，简体中文）。本次实际环境为 WSL2 下的 Ubuntu 24.04.2 LTS（x86_64），使用 GCC 13.3.0、Bash 5.2、GNU Make 4.3 和 GNU nano 7.2。

按当前登录用户执行操作，主目录为 /home/ovo155，对应题目示例中的 /home/osstudy。终端截图使用白色背景，时间按 Asia/Shanghai 记录。

三、实验内容

1—2．查看根目录，在当前用户主目录建立 A 和 bin

执行 mkdir A bin 创建两个目录。完整根目录清单及创建回显保存在 logs/01-root-and-directories.log；图 1 节选根目录前 8 行，便于查看本机目录结构。

```
$ pwd
/home/ovo155
$ uname -m
x86_64
$ ls -x / | sed -n '1,8p'
Docker     bin         bin.usr-is-merged  boot       dev
etc        home        init               lib        lib.usr-is-merged
lib64      lost+found  media              mnt        opt
proc       root        run                sbin       sbin.usr-is-merged
snap       srv         sys                tmp        usr
var        wslABpjmB   wslADnFgH          wslAGGFPJ  wslAJGFcG
wslAJdIKg  wslAKKILM   wslAMCakp          wslANkpBB  wslAbmDio
wslAeCceG  wslAegggo   wslAgLgKo          wslAgoBJh  wslAiAAom
$ ls -ld /bin /etc /home /dev /mnt
lrwxrwxrwx  1 root root    7 Apr 22  2024 /bin -> usr/bin
drwxr-xr-x 15 root root 3860 Nov 10 01:51 /dev
drwxr-xr-x 95 root root 4096 Nov 10 01:58 /etc
drwxr-xr-x  3 root root 4096 Jun 14  2025 /home
drwxr-xr-x  5 root root 4096 Jun 14  2025 /mnt
```

图 1　当前用户主目录、系统位数、根目录清单节选与五个目录属性

目录	意义
/bin	存放基本用户命令；本机为指向 usr/bin 的符号链接。
/etc	保存系统和服务的配置文件。
/home	保存普通用户的主目录，如/home/ovo155。
/dev	提供设备节点，程序通过它们访问设备。
/mnt	通常用作临时挂载文件系统的挂载点。

3—4 · 利用 nano 建立 sum.c，退出后在控制台显示源码

执行 nano A/sum.c，在编辑器中新建并保存程序。Ctrl+O 写入文件，回车确认文件名，Ctrl+X 退出。随后执行 cat A/sum.c，确认保存内容完整；完整控制台回显另见 screenshots/03-source-c.png 及同名日志。

程序用 fgets 读取输入，strtol 检查整型数及范围，利用 n(n+1)/2 计算 1 至 n 的和。乘法先提升为 long long，避免 int 溢出；本实现接受 0 至 2147483647，n=0 时输出 0，负数或非法输入报错。

```
  GNU nano 7.2                                  A/sum.c
/* 实验一：读取一个非负整型数 n，计算 1 + 2 + ... + n。 */
#include <ctype.h>
#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    char input[128];
    char *end;
    long n;
    long long sum;

    printf("Please enter an integer n (0..%d): ", INT_MAX);
    fflush(stdout);
    /* fgets + strtol 同时检查空输入、非法字符和超出整型范围。 */
    if (fgets(input, sizeof input, stdin) == NULL ||
        (strchr(input, '\n') == NULL && !feof(stdin))) {
        fprintf(stderr, "Input Error! A complete integer is required.\n");
        return EXIT_FAILURE;
    }
    errno = 0;
    n = strtol(input, &end, 10);
    if (end == input || errno == ERANGE || n < 0 || n > INT_MAX) {
        fprintf(stderr, "Input Error! n must be in 0..%d.\n", INT_MAX);
        return EXIT_FAILURE;
    }
    while (isspace((unsigned char)*end)) {
        ++end;
    }
    if (*end != '\0') {
        fprintf(stderr, "Input Error! Unexpected characters after n.\n");
        return EXIT_FAILURE;
    }
    /* 先提升到 long long，再相乘，避免 int 乘法溢出；n=0 时和为 0。 */
    sum = (long long)n * ((long long)n + 1) / 2;
    printf("1 + ... + %ld = %lld\n", n, sum);
    return EXIT_SUCCESS;
}




                        [ Wrote 41 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

图 2　GNU nano 中保存的完整 C 程序（41 行，含输入检查与注释）

源码文件：src/sum.c。输入 10 时应得到 10×11÷2=55；实际编译运行结果见下一页。

5—9 · 编译、复制、压缩、修改权限并删除 A

将 A/sum.c 编译链接为 A/sum，输入 10 验证结果为 55，再将可执行文件复制到 bin。用 tar 把 A 内的全部文件打包并 gzip 压缩到主目录 sum.tar.gz；-C A 表示先进入 A，末尾的点表示包含 A 中的全部条目。

```
$ gcc -std=c11 -Wall -Wextra -Wpedantic -Werror -O2 A/sum.c -o A/sum
$ printf '10\n' | ./A/sum
Please enter an integer n (0..2147483647): 1 + ... + 10 = 55
$ cp A/sum bin/
$ tar -czvf sum.tar.gz -C A .
./
./sum.c
./sum
$ chmod 755 sum.tar.gz
$ stat -c '%A %a %n' sum.tar.gz
-rwxr-xr-x 755 sum.tar.gz
$ rm -r -- A
$ test ! -e A && echo 'A has been deleted.'
A has been deleted.
$ printf '10\n' | ./bin/sum
Please enter an integer n (0..2147483647): 1 + ... + 10 = 55
```

图 3　严格编译、求和、复制、压缩、755 权限及删除 A 后的再次运行

操作	实际结果
gcc 严格编译	-std=c11 -Wall -Wextra -Wpedantic -Werror；未出现警告或错误。
A/sum 与 bin/sum 输入 10	两次输出均为 1 + ... + 10 = 55。
tar -czvf sum.tar.gz -C A .	归档含./、./sum.c 与./sum，生成在/home/ovo155。
chmod 755 sum.tar.gz	stat 显示-rwxr-xr-x 和 755。
rm -r -- A	test 确认 A 已删除；bin/sum 仍能独立运行。

755 由 7、5、5 组成：所有者为 rwx，所属组和其他用户均为 r-x，因此只有所有者具有写权限，符合题目要求。压缩包一般无需执行位，本实验按题目明确要求保留该位。

sum.tar.gz 及后续实验输出已复制到 evidence/home 中保存，原始终端命令与退出状态保存在 logs 中。

四、思考与实践

1．如何解压.tar.gz 文件？

先创建恢复目录，再执行 tar -xzvf sum.tar.gz -C restore-ex1。其中-x 解包，-z 使用 gzip，-v 显示条目，-f 指定压缩包。恢复后的 sum.c 与提交源码逐字节相同，恢复后的 sum 输入 10 仍输出 55。

```
$ mkdir restore-ex1
$ tar -xzvf sum.tar.gz -C restore-ex1
./
./sum.c
./sum
$ cmp restore-ex1/sum.c '/home/ovo155/Mycode/fzucourse/2601-fzu-OperatingSys-YaruSu/homework/Experimen
t/task1&2/src/sum.c' && echo 'Source matches.'
Source matches.
$ printf '10\n' | ./restore-ex1/sum
Please enter an integer n (0..2147483647): 1 + ... + 10 = 55
$ stat -c '%U:%G %A %n' restore-ex1/sum.c
ovo155:ovo155 -rw-r--r-- restore-ex1/sum.c
$ chown "$(id -un):$(id -gn)" restore-ex1/sum.c
$ stat -c '%U:%G %A %n' restore-ex1/sum.c
ovo155:ovo155 -rw-r--r-- restore-ex1/sum.c
$ lsblk -o NAME,TYPE,SIZE,MOUNTPOINTS | head -12
NAME TYPE   SIZE MOUNTPOINTS
sda  disk 388.6M
sdb  disk   186M
sdc  disk     4G [SWAP]
sdd  disk     1T /var/lib/docker
                 /mnt/wslg/distro
                 /
```

图 4　解压、源码比较、程序运行、所有者核对及块设备查看

2．如何修改文件所有者，修改后效果如何？

命令格式为 sudo chown 新用户:新组 文件；用 ls -l 或 stat 查看所有者、所属组。本次对练习文件执行 chown "$(id -un):$(id -gn)" restore-ex1/sum.c，将其设置为当前用户和组，前后均为 ovo155:ovo155。

若管理员把文件改属另一用户，文件的所有者身份随之变化；访问时按新的身份适用所有者、组或其他用户权限。chown 通常不直接改写普通 rwx 位，某些特殊权限位可能被系统清除。

3．如何挂载 U 盘等外部存储设备？

先用 lsblk -f 识别设备与分区，再创建挂载点并挂载，例如：

```
sudo mkdir -p /mnt/usb
sudo mount /dev/sdX1 /mnt/usb
findmnt /mnt/usb
sudo umount /mnt/usb
```

/dev/sdX1 为示意占位，实际须替换为确认过的分区。本次未接入用于实验的 U 盘，故只检查了块设备，不把上述挂载步骤记作已执行操作；WSL 下还可能需要先由宿主机附加设备。

4．尝试 Ubuntu 的图形化工具

在当前 Ubuntu 环境中实际打开下列图形程序，分别查看文件、浏览本地页面、阅读 Word 模板并尝试图形软件包管理器。图 5 直接截取运行中的窗口，软件包管理器以普通用户模式启动。

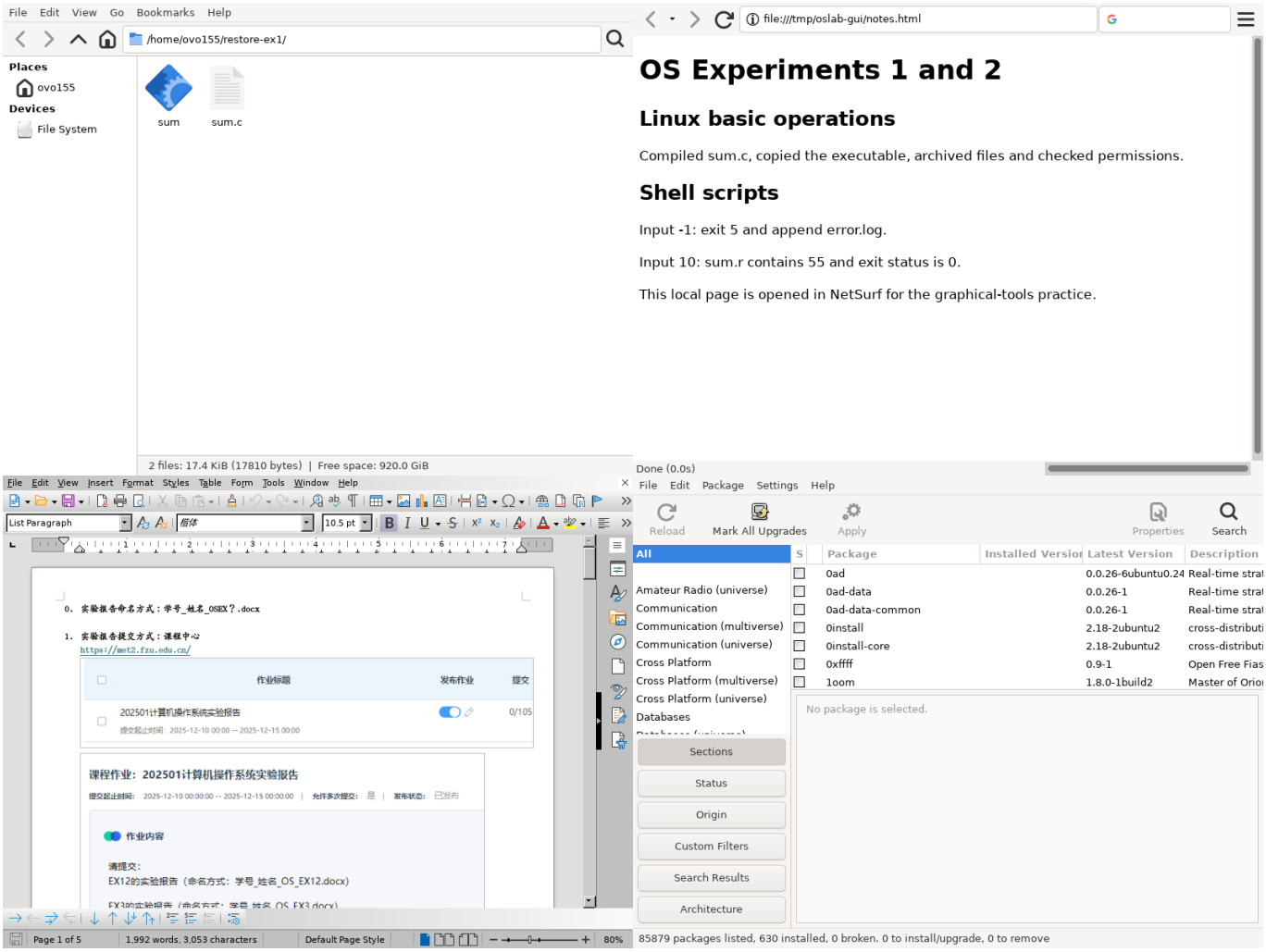


图 5　文件管理器、浏览器、文字处理工具和图形软件包管理器

工具	操作与观察
Thunar 文件管理器	打开 restore-ex1 目录，查看解压得到的 sum 与 sum.c。
NetSurf 浏览器	打开本地实验说明 HTML，观察浏览器显示页面内容。
LibreOffice Writer	打开课程提供的实验要求.docx，查看模板及其分页。
Synaptic 软件包管理器	作为软件中心类工具的替代，尝试普通用户模式下的包管理界面；未执行安装、删除或更新。

本实验把同一组文件从命令行和图形界面分别查看：命令行适合记录和重复执行，图形工具便于直观浏览目录与文档。编译、归档、权限和解压结果均有实际输出可核对。

《计算机操作系统》上机实验报告 EX2

实验 2 Linux 的基本操作（续）

学号：待填写　　姓名：待填写　　班级：待填写　　实验日期：2026 年 11 月 10 日

一、实验目的

1．了解管道和重定向。

2．熟悉基本的 Linux 脚本的编写。

3．熟悉基本的 Makefile 脚本的编写（选做内容一并完成）。

实验环境同实验一：Ubuntu 24.04.2 LTS，Bash 5.2、GNU Make 4.3。按模板第 3 页保留“实验目的—实验内容—思考与实践”结构。

二、实验内容

1—3．生成主目录清单，管道筛选根目录日期，再合并文件

```
ls -l > list1.txt
LC_ALL=C ls -lA --time-style=+%m-%d / | awk '$6 == "04-16"' > list2.txt
cat list1.txt list2.txt > list.txt
```

```
$ ls -l > list1.txt
$ LC_ALL=C ls -lA --time-style=+%m-%d / | awk '$6 == "04-16"' > list2.txt
$ cat list1.txt list2.txt > list.txt
$ cat list1.txt
total 48
drwxr-xr-x 5 ovo155 ovo155 4096 Sep 12 03:15 Mycode
drwxr-xr-x 2 ovo155 ovo155 4096 Nov 10 02:12 bin
-rw-r--r-- 1 ovo155 ovo155    3 Nov 10 02:13 custom result.txt
-rw-r--r-- 1 ovo155 ovo155   42 Nov 10 02:12 error.log
drwxr-xr-x 4 ovo155 ovo155 4096 Apr 28  2026 go
-rw-r--r-- 1 ovo155 ovo155 2517 Nov 10 02:12 list.txt
-rw-r--r-- 1 ovo155 ovo155    0 Nov 10 02:14 list1.txt
-rw-r--r-- 1 ovo155 ovo155    0 Nov 10 02:12 list2.txt
drwxr-xr-x 3 ovo155 ovo155 4096 May 17 20:20 logs
drwxr-xr-x 2 ovo155 ovo155 4096 Nov 10 02:12 restore-ex1
-rw-r--r-- 1 ovo155 ovo155    3 Nov 10 02:12 sum.r
-rwxr-xr-x 1 ovo155 ovo155 1584 Nov 10 02:12 sum.sh
-rwxr-xr-x 1 ovo155 ovo155 3974 Nov 10 02:12 sum.tar.gz
-rwxr-xr-x 1 ovo155 ovo155  231 Nov 10 02:03 sum_named.sh
$ cat list2.txt
$ wc -l list1.txt list2.txt list.txt
  15 list1.txt
   0 list2.txt
  15 list.txt
  30 total
$ cmp list.txt <(cat list1.txt list2.txt) && echo 'Concatenation verified.'
Concatenation verified.
```

图 6　文件重定向、4 月 16 日日期筛选、行数统计及合并验证

本次 list1.txt 为 15 行，list2.txt 为 0 行，list.txt 为 15 行。根目录一级条目没有符合 4 月 16 日的修改日期，因此 list2.txt 为空，这是本机的真实筛选结果。cmp 确认 list.txt 与两份清单顺序拼接后的内容完全一致。

用 LC_ALL=C 固定字段格式，--time-style=+%m-%d 固定月份和日期，awk 按第 6 列精确比较“04-16”，不限定年份；-A 包含隐藏条目但排除.和..。这里筛选根目录直接包含的条目，不递归遍历。重定向先创建 list1.txt，所以它自身也会出现在清单中。

4．利用 nano 编写求和脚本 sum.sh

执行 nano sum.sh，保存后退出，并用 chmod +x sum.sh 赋予执行权限。脚本通过 read 读取 n，用条件判断验证输入；非法输入用 date 取得当前时间，追加到 error.log 并 exit 5；有效输入将求和结果重定向到 sum.r。

```
  GNU nano 7.2                                  sum.sh
#!/usr/bin/env bash
# 实验二：默认写入 sum.r；--choose-output 实现“先询问输出文件名”。
# 输入限制为 32 位正整数，使 Bash 的 64 位整数运算不会溢出。
set -u

choose_output=0
if [[ $# -eq 1 && $1 == --choose-output ]]; then
    choose_output=1
elif [[ $# -ne 0 ]]; then
    printf 'Usage: %s [--choose-output]\n' "$0" >&2
    exit 2
fi

input_error() {
    local message
    message="$(date '+%Y-%m-%d %H:%M:%S') Input Error!  n = $raw"
    # >> 保留以前的错误记录；先检查日志写入结果。
    if ! printf '%s\n' "$message" >> error.log; then
        exit 1
    fi
    printf '%s\n' "$message" >&2
    exit 5
}

printf 'Please enter a positive integer n: '
raw=''
IFS= read -r raw || [[ -n $raw ]] || input_error
[[ $raw =~ ^[+]?[0-9]+$ ]] || input_error
digits=${raw#+}
# 去掉前导零后再检查范围，避免把 08 当成八进制或让超长数字回绕。
while [[ ${#digits} -gt 1 && $digits == 0* ]]; do
    digits=${digits#0}
done
[[ ${#digits} -le 10 ]] || input_error
[[ $digits -gt 0 && $digits -le 2147483647 ]] || input_error
n=$((10#$digits))
sum=$((n * (n + 1) / 2))

output='sum.r'
if ((choose_output)); then
    printf 'Output filename: '
    if ! IFS= read -r output || [[ -z $output ]]; then
        printf 'Output Error! A filename is required.\n' >&2
        exit 1
    fi
fi
# 引号保证文件名中含空格时仍是一个路径；写入失败不得报告成功。
if ! printf '%s\n' "$sum" > "$output"; then
    exit 1
fi
printf '1 + ... + %s = %s; saved to %s\n' "$n" "$sum" "$output"
exit 0


                        [ Wrote 52 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

图 7　nano 中保存的 sum.sh；完整可编辑源码见 src/sum.sh

使用>>追加日志，使用>写入本次结果。输入限制为 32 位正整数，求和由 Bash 64 位整数运算完成；先处理前导零，避免 08 被当作八进制。--choose-output 分支供思考与实践使用，默认行为仍写入 sum.r。

5．分别以-1 和 10 为 n 运行，核对日志、结果及退出码

```
$ chmod +x sum.sh sum_named.sh
$ printf '%s\n' -1 | ./sum.sh; code=$?; printf 'exit status = %s\n' "$code"
Please enter a positive integer n: 2026-11-10 02:12:59 Input Error！ n = -1
exit status = 5
$ cat error.log
2026-11-10 02:12:59 Input Error！ n = -1
$ if [ -e sum.r ]; then cat sum.r; else echo 'sum.r does not exist.'; fi
sum.r does not exist.
$ printf '10\n' | ./sum.sh; code=$?; printf 'exit status = %s\n' "$code"
Please enter a positive integer n: 1 + ... + 10 = 55; saved to sum.r
exit status = 0
$ cat sum.r
55
$ cat error.log
2026-11-10 02:12:59 Input Error！ n = -1
```

图 8　n=-1 返回 5 并追加日志；n=10 返回 0 并把 55 写入 sum.r

输入	error.log	sum.r	退出码
-1	新增当前时间及 Input Error！ n = -1	首次非法运行后不存在	5
10	保留上一条错误记录，不新增错误	内容为 55（带换行）	0

每次运行后立即执行 code=$?保存退出状态，再查看文件，避免后续 cat 等命令覆盖$?。错误行满足题目要求的“<当前时间> Input Error！ n = <输入值>”格式。本次截图中的时间为 2026-11-10，按中国标准时间记录。

三、思考与实践

1．修改脚本，在输出结果前询问文件名

在 sum.sh 加入--choose-output 选项：先验证 n 并求和，再 read 读取输出文件名。sum_named.sh 调用该分支；文件路径始终加双引号，因而带空格的文件名也能正确处理。

```
$ printf '10\ncustom result.txt\n' | ./sum_named.sh; code=$?; echo "exit status = $code"
Please enter a positive integer n: Output filename: 1 + ... + 10 = 55; saved to custom result.txt
exit status = 0
$ cat 'custom result.txt'
55
$ cat sum.r
55
$ cat error.log
2026-11-10 02:12:59 Input Error！ n = -1
```

图 9　输入 10 及 custom result.txt，生成的自选结果文件为 55

完整包装脚本见 src/sum_named.sh。写入失败返回 1，不输出保存成功；该行为及空格文件名都已验证。

2．查阅资料，尝试编写 Makefile

阅读提供的《扩展阅读——跟我一起写 Makefile》，编写本目录的 Makefile。默认目标 all 生成 build/sum；build/sum 依赖 build/sum.o，后者依赖 src/sum.c，分别演示编译和链接；clean、check 声明为 .PHONY。命令行前使用真正的 TAB，$<表示第一个依赖，$@表示当前目标。

```
$ make clean
rm -f build/sum.o build/sum
$ make
mkdir -p build
gcc  -std=c11 -Wall -Wextra -Wpedantic -Werror -O2 -c src/sum.c -o build/sum.o
gcc  build/sum.o  -o build/sum
$ make
make: Nothing to be done for 'all'.
$ make -q; code=$?; echo "make -q status = $code"
make -q status = 0
$ make -n -W src/sum.c
mkdir -p build
gcc  -std=c11 -Wall -Wextra -Wpedantic -Werror -O2 -c src/sum.c -o build/sum.o
gcc  build/sum.o  -o build/sum
```

图 10　首次构建、重复 make、make -q 及源码变化时的重建计划

make clean 后首次 make 执行编译和链接；第二次 make 提示无需更新；make -q 返回 0，表示目标已是最新。make -n -W src/sum.c 模拟源码更新，显示应重新编译和链接，但不实际改动源码时间。

验证结果与实验记录

```
$ tail -18 logs/verification.log
PASS 14: n=10 writes 55, exits 0 and keeps error.log unchanged
PASS 15: invalid input appends rather than overwrites and preserves prior result
PASS 16: shell rejects 'abc'
PASS 17: shell rejects '1.5'
PASS 18: shell rejects ''
PASS 19: shell rejects '2147483648'
PASS 20: shell rejects '999999999999999999'
PASS 21: shell decimal/range handling n=08
PASS 22: shell decimal/range handling n=+10
PASS 23: shell decimal/range handling n=00010
PASS 24: shell decimal/range handling n=65536
PASS 25: shell decimal/range handling n=2147483647
PASS 26: extension asks for a filename and supports spaces
PASS 27: output failure returns an error
PASS 28: pipeline matches April 16 across years and excludes April 17
PASS 29: list concatenation is byte-exact
PASS 30: no April 16 entries is a successful empty result
All 30 checks passed.
```

图 11　自动验证的末段输出：All 30 checks passed.

30 项检查覆盖 C 程序边界、脚本必测输入、错误日志追加、已有结果保留、前导零、数值范围、自选输出和写入失败，以及不同年份的 4 月 16 日筛选、4 月 17 日排除、无匹配与文件合并。样例日期测试只在临时目录中进行，不修改根目录或报告截图中的系统时间。

依据：实验要求.docx 第 2—3 页模板；上机作业.pptx 第 2—9 页；Linux 基本操作知识点提纲、Linux 基础教程、命令行与 shell 脚本编程宝典中的重定向/管道/退出状态说明，以及 Makefile 扩展阅读。原始输出见 logs，结果文件见 evidence/home。